

STREAMS CODING QUESTIONS

1. Write a Java program to calculate the average of a list of integers using streams.

```
double average = nums.stream()
    .mapToDouble(Integer :: doubleValue)
    .average()
    .orElse(0.0);
```

```
double sum = nums.stream()
    .mapToDouble(Integer :: doubleValue)
    .sum();
```

2. Write a Java program to convert a list of strings to uppercase or lowercase using streams.

```
List<String> list = strs.stream()
    .map(String :: toUpperCase)
    .collect(Collectors.toList());
```

3. Write a Java program to calculate the sum of all even, odd numbers in a list using streams.

```
int evenSum = nums.stream()
    .mapToInt(Integer :: intValue)
    .filter(n -> n%2 == 0)
    .sum();

int oddSum = nums.stream()
    .mapToInt(Integer :: intValue)
    .filter(n -> n%2 != 0)
    .sum();
```

4. Write a Java program to remove all duplicate elements from a list using streams.

```
Set<Integer> set = new HashSet<>();

List<Integer> res = nums.stream()
    .filter(n -> !set.add(n))
    .collect(Collectors.toList());
```

5. Write a Java program to count the number of strings in a list that start with a specific letter using streams.

```
16         long count = nums.stream()
17             .map(s -> s+"")
18             .filter(s -> s.startsWith("1"))
19             .count();
20
```

6. Write a Java program to sort a list of strings in alphabetical order, ascending and descending using streams.

```
List<String> res = strs.stream()
    .sorted(Comparator.reverseOrder())
    .collect(Collectors.toList());
```

```
List<String> res = strs.stream()
    .sorted()
    .collect(Collectors.toList());
```

7. Write a Java program to find the maximum and minimum values in a list of integers using streams.

```
int max = nums.stream()
    .max(Integer :: compare)
    .orElse(null);
```

```
int min = nums.stream()
    .min(Integer :: compare)
    .orElse(null);
```

8. Write a Java program to find the second smallest and largest elements in a list of integers using streams.

```
int secondSmallest = nums.stream()
    .distinct()
    .sorted()
    .skip(1)
    .findFirst()
    .orElse(null);
```

```
int secondLargest = nums.stream()
    .distinct()
    .sorted(Comparator.reverseOrder())
    .skip(1)
    .findFirst()
    .orElse(null);
```

SET 2 – CODING QUESTIONS ON STREAMS

1. Filter out even numbers from a list of integers.

```
List<Integer> res = nums.stream()
    .filter(n -> n%2 == 0)
    .collect(Collectors.toList());
```

2. Find the sum of all elements in a list of integers.

```
int res = nums.stream()
    .mapToInt(Integer :: intValue)
    .sum();
```

3. Calculate the average of a list of floating-point numbers.

```
double res = nums.stream()
    .mapToDouble(Integer :: doubleValue)
    .average()
    .orElse(0.0);
```

4. Count the number of distinct elements in a list.

```
double res = nums.stream()
    .filter(n -> set.add(n))
    .count();
```

5. Convert a list of strings to uppercase.

```
List<String> res = strs.stream()
    .map(String :: toUpperCase)
    .collect(Collectors.toList());
```

6. Remove duplicate strings from a list.

```
List<String> res = strs.stream()
    .filter(s -> set.add(s))
    .collect(Collectors.toList());
```

7. Find the maximum element in an array of integers using streams.

```
int res = nums.stream()
    .max(Integer :: compare)
    .orElse(null);
```

8. Find the minimum element in a list of doubles using streams.

```
int res = nums.stream()
    .min(Integer :: compare)
    .orElse(null);
```

9. Sort a list of strings in ascending order.

```
List<String> res = strs.stream()
    .sorted()
    .collect(Collectors.toList());
```

10. Group a list of people by their age.

11. Find the longest string in a list of strings.

```
String res = strs.stream()
    .max((s1, s2) -> Integer.compare(s1.length(), s2.length()))
    .orElse(null);
```

12. Compute the product of all elements in a list of integers.

```
long res = nums.stream()
    .reduce(1, (a,b) -> a*b);
```

13. Check if all elements in a list of booleans are true.

```
15      boolean res = bool.stream()
16      .allMatch(Boolean::booleanValue);
17
input
List of booleans: [true, false, true, true, true]
false

15      boolean res = bool.stream()
16      .allMatch(Boolean::booleanValue);
17
input
List of booleans: [true, true, true, true]
true
```

14. Find the first non-empty string in a list of strings.

```
Optional<String> res = strs.stream()
    .filter(s -> !s.isEmpty())
    .findFirst();
```

15. Concatenate all strings in a list into a single string.

```
String res = strs.stream()
    .collect(Collectors.joining());
```

16. Extract all unique words from a list of sentences.

```
Set<String> uniqueWords = strs.stream()
    .flatMap(sentence -> Arrays.stream(sentence.split("\\s+")))
    .map(word -> word.replaceAll("[^a-zA-Z]", "").toLowerCase())
    .filter(word -> !word.isEmpty())
    // .distinct() --> no need of it as set doesn't allow duplicates
    .collect(Collectors.toSet());
```

17. Calculate the factorial of a given integer using streams.

```
int n = 10;

long factorial = IntStream.rangeClosed(1, n)
    .reduce(1, (a, b) -> a * b);
```

18. Find the top 5 highest-scoring students from a list of student objects.

19. Remove all null values from a list.

```
List<String> listWithoutNulls = strs.stream()
    .filter(element -> element != null)
    .collect(Collectors.toList());
```

20. Check if there are any prime numbers in a list of integers.

```

import java.util.Arrays;
import java.util.List;

public class CheckPrimeNumber {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(2, 4, 5, 8, 9, 11, 13);

        // Check if there are any prime numbers in the list using streams
        boolean hasPrime = numbers.stream()
            .anyMatch(CheckPrimeNumber::isPrime);

        if (hasPrime) {
            System.out.println("The list contains prime numbers.");
        } else {
            System.out.println("The list does not contain prime numbers.");
        }
    }

    public static boolean isPrime(int n) {
        // Check if the number is less than or equal to 1.
        // Numbers less than or equal to 1 are not prime.
        if (n <= 1) {
            return false;
        }

        // Check if the number is 2 or 3.
        // Both 2 and 3 are prime numbers.
        if (n <= 3) {
            return true;
        }

        // Check if the number is divisible by 2 or 3.
        // If it is divisible by 2 or 3, it's not prime.
        if (n % 2 == 0 || n % 3 == 0) {
            return false;
        }

        // Check for prime numbers using a 6k +/- 1 pattern.
        // This optimization reduces the number of divisions needed.
        // We start from 5 and increment by 6.
        // We only need to check divisors up to the square root of n.
        for (int i = 5; i * i <= n; i += 6) {
            if (n % i == 0 || n % (i + 2) == 0) {
                // If the number is divisible by i or (i + 2), it's not prime.
                return false;
            }
        }

        // If none of the above conditions were met, the number is prime.
        return true;
    }
}

```